REPORT

Reviewer report: andialbrecht/sqlparse#944: fix: avoid

ValueError on malformed CASE with reindent_aligned

Run sqlparse-944, tested 2026-11-06 11:29 IST by ops/daily_run.py (route source-pair). Proof-of-Change

commit 4050401d30fe09fa6a04495c8f41d7203ddf9de8. Report generated 2026-11-06 11:30 IST by

reports/pr_report.py.

Baseline: full (test comparison + claim probes + coverage gate + claim flagging (differential probes shown,

informational)).

VERIFIED

Recommended action: Approve Confidence: High

The PR's tests behave the same or better than on main, in all 3 runs per side. Every check passed and every

claim cites a log line.

Advice only: Proof-of-Change never blocks a merge and never posts on GitHub.

Summary in plain English

• What it says it does: fix: avoid ValueError on malformed CASE with reindent_aligned

(andialbrecht/sqlparse#944).

• What we observed: 175 test(s) ran on main, 177 on the PR; 2 new test(s) pass, 177 unchanged (2 of them

skipped on both sides); probe: claim(s) C2 proven.

• What you should do: Nothing to fix. You can approve.

Demo helper

These notes are background from the console and the daily check. They do not change the verdict.

If setup fails, the helper retries the install up to 2 times. If a test still can't be verified because something is

missing or fails to import, it looks up the cause, installs the fix on both sides and reruns that step, up to 3 tries.

• Setup check: GitHub token: present.

• Setup check: Nebius token: present.

• Setup check: free memory 6.9 GB (of 15.6 GB).

• Setup check: free disk 94.7 GB.

• Demo helper: git is installed.

• Demo helper: python3 is installed.

• Demo helper: uv is installed.

• Looked up with Tavily: [redacted] results

• Query: sqlparse python setup install instructions https://github.com/andialbrecht/sqlparse site:github.com/andialbrecht/sqlparse

• Query: sqlparse python documentation install https://github.com/andialbrecht/sqlparse site:github.com/andialbrecht/sqlparse

• Query: sqlparse python install requirements https://pypi.org/project/sqlparse/ https://github.com/andialbrecht/sqlparse site:pypi.org

• Source: sqlparse/pyproject.toml at master · andialbrecht/sqlparse · GitHub — https://github.com/andialbrecht/sqlparse/blob/master/pyproject.toml

• Source: sqlparse · PyPI — https://pypi.org/project/sqlparse/

• Source: GitHub - andialbrecht/sqlparse: A non-validating SQL parser module for Python · GitHub — https://github.com/andialbrecht/sqlparse

• Source: sqlparse/Makefile at master · andialbrecht/sqlparse · GitHub — https://github.com/andialbrecht/sqlparse/blob/master/Makefile

• Source: sqlparse/.github/workflows/python-app.yml at master · andialbrecht/sqlparse · GitHub — https://github.com/andialbrecht/sqlparse/blob/master/.github/workflows/python-app.yml

• Source: sqlparse/.readthedocs.yaml at master · andialbrecht/sqlparse · GitHub — https://github.com/andialbrecht/sqlparse/blob/master/.readthedocs.yaml

• Setup plan: the same packages on main and pr.

• Setup plan: install build (listed in the dev extra) on main and pr.

• Setup plan: Python 3.12: Python 3.12 already meets >=3.10

• Installed build 1.6.1 from pyproject (listed in the dev extra) on main and pr.

Claims vs reality

| # | Claim (source) | Status | What we observed | Log line

| C2 | Existing behavior is kept: tests that pass on main still pass (implied by every PR) | proven | no test that passes on main fails on the PR (177 unchanged, 3 runs per side) | pytest-pr-1.log:9 «176 passed, 2 xfailed, 1 xpassed in 0.76s»

| P-C2 | Valid CASE alignment behavior remains unchanged after the fix. (PR description) | proven | probe model-p1, c2.empty_case_arms: same on both sides: "CASE WHEN 1 THEN\n END" | probe-model-p1-pr-1.jsonl:1 «"observation": "c2.empty_case_arms"»

Citation check (PASS): 2 citation(s), every one re-read from its log file and matched.

What changed in behavior

| What | Count | Examples

| Tests that actually ran (main / PR, fewest of the runs) | 175 / 177 | | Tests unchanged | 177 | | New tests that pass | 2 | | Tests that pass on main but fail on the PR | 0 | | Tests that are new in the PR and fail | 0 | | Tests that ran on main but are missing on the PR side | 0 | | Tests that changed result in another way (for example passed -> skipped) | 0 | | Tests that gave different results between the 3 runs of the same side (flaky) | 0 | | Tests that fail on main and pass on the PR | 0 | | Tests that are new or changed but did not run because an optional module is missing | 0 | | Tests that were re-parametrized (same test, new parameter ids; not counted as removed) | 0 | | Test modules that fail to load on the PR side only (their tests no longer run) | 0 | | Test modules that fail to load on main only (no baseline for their tests) | 0 | -

| Test modules that fail to load on both sides (their tests did not run) | 0 | | Tests that fail identically on both sides (already broken on main) | 0 | -

Probes

Model-written probes: model provider: nebius (Token Factory, live).

A probe is a small deterministic script run on both sides in the same environments as the tests, 3 alternating runs

per side. It prints JSON observations; plain code (poc/diff.py) compares them. A difference that no claim of the

PR covers needs attention. Every probe was frozen (content sha256) before any PR-side code ran, and checked

against that freeze before each PR-side run. Only a live model probe is blind; fixed probes and recorded model

fixtures are known-answer cases.

model-p1 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-p1.py, frozen 81f03228455a

at 2026-11-06T06:00:36.876385+00:00): Check that valid CASE alignment is unchanged

• P-C2 (PR description): Valid CASE alignment behavior remains unchanged after the fix. Expects the covered

observations to stay the same.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c2.empty_case_arms | P-C2 | as claimed | same on both sides: "CASE WHEN 1 THEN\n END" |

probe-model-p1-main-1.jsonl:1 «"observation": "c2.empty_case_arms"»
probe-model-p1-pr-1.jsonl:1

«"observation": "c2.empty_case_arms"»

| c2.nested | P-C2 | as claimed | same on both sides: "SELECT CASE WHEN EXISTS (SELECT 1 FROM t

WHERE a = b) THEN 1\n ELSE 0\n ... | probe-model-p1-main-1.jsonl:2 «"observation":

"c2.nested"»
probe-model-p1-pr-1.jsonl:2 «"observation": "c2.nested"»

| c2.simple | P-C2 | as claimed | same on both sides: "CASE WHEN x = 1 THEN 'a'\n ELSE 'b'\n END" |

probe-model-p1-main-1.jsonl:3 «"observation": "c2.simple"»
probe-model-p1-pr-1.jsonl:3 «"observation":

"c2.simple"»

| c2.with_else | P-C2 | as claimed | same on both sides: "CASE WHEN x = 1 THEN 'a'\n WHEN x = 2 THEN 'b'\n

ELSE 'c'\n ... | probe-model-p1-main-1.jsonl:4 «"observation": "c2.with_else"»
probe-model-p1-pr-1.jsonl:4

«"observation": "c2.with_else"»

Citation check (PASS): 8 citation(s), every one re-read from its log file and matched.

Was the changed code exercised?

Coverage: MEASURED by coverage.py on the PR side (1.9 s run).

Coverage gate (python, confidence normal): 8 of 13 changed executable line(s) ran (62%).

| Changed file | Changed executable lines | Of those, ran

| sqlparse/filters/aligned_indent.py | 13 | 8

How this is measured: one extra, untimed run of the PR side's tests under coverage.py. Changed lines come

from comparing the tested main and PR files. Compiled code (C, C++, Cython) is invisible to coverage.py; such a

change only counts as verified when the PR adds or changes a test for that module.

What went right

• Tested 51894597437c of the PR against main (source-pair route).

• 3 alternating runs per side: main 3, PR 3 of 3.

• pytest exit codes explained: all runs exited 0.

• Changed code was exercised (coverage gate): 8 of 13 changed executable line(s) ran (62%).

• Probes ran on both sides: 1 probe(s), 3 alternating runs per side, every run printed its observations.

• Single-thread environment: OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0.

• Machine type recorded: local x86_64 8 cpus.

• Nothing posted upstream: GitHub was only read.

• Known-answer replay (fresh re-run): replayed 2026-10-10 06:30:21 IST, harness 1.0.

• Every claim cites a real log line: 2 citation(s), every one re-read from its log file and matched.

• Every probe observation cites its output line: 8 citation(s), every one re-read from its log file and matched.

What went wrong and how to fix it

Nothing went wrong.

Evals and guardrails

| Check | Result | Detail

| 3 alternating runs per side | PASS | main 3, PR 3 of 3

| pytest exit codes explained | PASS | all runs exited 0

| Changed code was exercised (coverage gate) | PASS | 8 of 13 changed executable line(s) ran (62%)

| Probes ran on both sides | PASS | 1 probe(s), 3 alternating runs per side, every run printed its observations

| Single-thread environment | PASS | OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0

| Machine type recorded | PASS | local x86_64 8 cpus

| Nothing posted upstream | PASS | GitHub was only read

| Known-answer replay (fresh re-run) | PASS | replayed 2026-10-10 06:30:21 IST, harness 1.0

| Every claim cites a real log line | PASS | 2 citation(s), every one re-read from its log file and matched

| Every probe observation cites its output line | PASS | 8 citation(s), every one re-read from its log file and

matched

Checklist

• I read the verdict and the plain-English summary.

• Every 'false' or 'unmentioned change' row has a log line I can open.

• Nothing under 'What went wrong' is a run problem (those mean Re-run, not Reject).

• I recorded my decision (see below).

Decision recorded: none yet. To record one, run make decide

REPORT=poc-main/reports/daily/2026-11-06-check-1129/sqlparse-944 DECISION=approve (or

reject, approve_with_notes, re_run).

Rerun command

make check PR=https://github.com/andialbrecht/sqlparse/pull/944

Evidence links

• Pull request andialbrecht/sqlparse#944

• record.json

• pr.diff

• setup-main.log

• setup-pr.log

• git.log

• pytest-main-1.log

• pytest-pr-1.log

• pytest-main-2.log

• pytest-pr-2.log

• pytest-main-3.log

• pytest-pr-3.log

• junit-main-1.xml

• junit-pr-1.xml

• coverage-pytest.log

• coverage-pr.json

• probe-model-p1-main-1.jsonl

• probe-model-p1-pr-1.jsonl

• main run 1 result: pytest-main-1.log:9 «174 passed, 2 xfailed, 1 xpassed in 0.81s»

• pr run 1 result: pytest-pr-1.log:9 «176 passed, 2 xfailed, 1 xpassed in 0.76s»

Glossary

| Term | Meaning

| main | The library as it is today, without the PR.

| PR | Pull request: a proposed change to the library.

| run | One full pass of the selected tests. We do 3 per side, alternating main and PR.

| flaky | A test that gives different results when nothing changed; it is ignored as evidence.

| log line | A line in a pytest-*.log file: the raw output the verdict is based on.

| route | How the two sides were built: nightly+patch, source-build or source-pair (see docs/OPERATIONS.md).

| changed lines covered | How many of the PR's changed, executable Python lines ran during the PR side's tests.

| re-parametrized | A test whose parameter ids changed (renamed cases); the new ids are judged on their own.

rebuilt from cleaned text

Written 2026-11-06 11:31 IST